
AGENDA 9

**Modelos de
Desenvolvimento de
Sistemas.**

SEAD-Superintendência de Ensino e Pesquisa nas
modalidades EaD e Aberta

GOVERNO DO ESTADO DE SÃO PAULO
EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS
TECNOLOGIA DA INFORMAÇÃO I

Expediente

Autores:

José Mendes da Silva Neto

Kelly Cristiane de Oliveira Dal Pozzo

Revisão Técnica:

Tiago Rogério Pertile

São Paulo – SP, 2026

MOMENTO REFLEXÃO



Antes de iniciar seus estudos, pense: como um sistema pode realmente ajudar uma pessoa ou uma empresa se não for planejado levando em conta suas necessidades reais? Refletir sobre a importância de **entender exatamente o que o usuário espera e precisa é o primeiro passo para criar soluções tecnológicas úteis e eficazes.** Você já percebeu como ouvir, analisar e coletar informações corretas faz toda a diferença no sucesso de um projeto? Muitas vezes, o segredo para transformar uma ideia em um sistema que realmente funciona está na atenção aos detalhes do que o cliente precisa — e não apenas na tecnologia empregada.



Imagem 1: Reunião para levantamento e compreensão das necessidades.

O que, para você, faz a diferença entre um sistema que resolve de verdade e outro que só complica a vida de quem usa?

PORQUE APRENDER?



Imagem 2: Colaboração na Definição das Necessidades do Sistema.

Você já percebeu como, no desenvolvimento de sistemas, não basta apenas saber programar ou conhecer as tecnologias mais recentes? O verdadeiro diferencial está em enxergar além do código: é conseguir entender, com clareza, o que o cliente realmente precisa e transformar essas necessidades em soluções concretas.

Quando você domina o levantamento de requisitos, vai além de executar tarefas: passa a compreender a lógica por trás de cada demanda, identifica prioridades e antecipa possíveis desafios que podem surgir durante o projeto. Não se trata apenas de fazer

o que pedem, mas de entender o contexto, questionar, propor melhorias e evitar surpresas no meio do caminho. Ao aprender a coletar dados, ouvir o usuário e representar essas necessidades de forma visual, por exemplo, usando o **Diagrama de Casos de Uso da UML** você desenvolve a capacidade de traduzir situações do mundo real em sistemas digitais eficientes. Esse cuidado contribui para reduzir falhas e retrabalho e aumenta as chances de entregar uma solução que atenda às necessidades dos usuários. Com esses conhecimentos, você se torna capaz de planejar sistemas que realmente fazem a diferença, entregando soluções alinhadas às expectativas e necessidades de quem vai usá-las. O domínio dessas ferramentas e métodos é o que transforma um desenvolvedor comum em um profissional valorizado no mercado de tecnologia.



PARA COMEÇAR O ASSUNTO

Jéssica está iniciando seu trabalho como desenvolvedora em uma pequena empresa de tecnologia e acaba de receber seu primeiro grande desafio: criar um sistema para uma loja de bairro que deseja modernizar seus processos. Ao conversar com o dono, ela percebe rapidamente que ele não consegue detalhar exatamente o que precisa, relatando problemas com estoque, dificuldades para encontrar informações dos clientes e até de vendas perdidas por falta de controle.

Nos dias seguintes, Jéssica busca compreender melhor a rotina da loja: conversa com o proprietário e com os funcionários, observa o funcionamento do estabelecimento e analisa as planilhas utilizadas no dia a dia. A cada conversa, surgem novas informações e, com elas, dúvidas: será que o sistema deve priorizar o controle de estoque ou o relacionamento com os clientes? O que é mais urgente resolver? Quais funcionalidades realmente farão diferença no dia a dia daquela loja?

Em uma reunião, Jéssica desabafa:

— Parece simples criar um sistema, mas entender exatamente o que o cliente precisa é muito mais complicado do que eu imaginava...

Foi então que as perguntas começaram a surgir:

- Como saber quais funções o sistema deve ter sem um levantamento detalhado das necessidades do dono da loja?
- O que pode acontecer se alguma informação importante for ignorada ou mal interpretada?
- Como transformar as demandas do cliente em requisitos claros e compreensíveis para toda a equipe de desenvolvimento?
- Que ferramentas podem ajudar a organizar, visualizar e validar tudo isso antes que o sistema comece a ser programado?

- Qual o papel do Diagrama de Casos de Uso nesse processo de descoberta e documentação?



MERGULHANDO NO TEMA

Essas dúvidas mostram que, muitas vezes, o desafio não está apenas em programar, mas em conseguir traduzir o que acontece no dia a dia da loja em requisitos bem definidos e representados. Para entender melhor essa situação e descobrir caminhos para criar sistemas que realmente resolvem problemas, vamos mergulhar no levantamento de requisitos e na modelagem com casos de uso.

Em empresas de tecnologia que desenvolvem soluções para o varejo, como aquela em que Jéssica trabalha, o ponto de partida para qualquer projeto de software é entender o que realmente precisa ser construído. Imagine que, antes de qualquer linha de código, toda a equipe se reúne para discutir o que o sistema deve resolver, quais problemas dos usuários precisam ser atacados e quais expectativas precisam ser atendidas. É nesse momento que o conceito de requisito se torna peça central na construção de sistemas eficientes. Mas nem sempre foi assim. Em projetos mais antigos, era comum que desenvolvedores partissem direto para a programação, confiando apenas em conversas informais ou em listas pouco detalhadas de funcionalidades. Com o tempo, percebeu-se que a falta de clareza sobre o que o sistema deveria fazer era uma das principais causas de fracasso em projetos de software. Hoje, o levantamento de requisitos é visto como uma etapa estratégica, capaz de evitar retrabalho, desperdício de recursos e insatisfação do cliente. A Análise de Sistemas funciona como uma ponte vital entre o universo dos negócios e o mundo da tecnologia. O analista atua como um tradutor: ele ouve as dores, processos e desejos dos usuários e os transforma em especificações técnicas que os desenvolvedores podem seguir. Sem essa etapa de planejamento e investigação, corre-se o risco de construir uma ferramenta tecnicamente perfeita, mas que não atende às necessidades reais de quem vai utilizá-la.

Dentro desse contexto, a Coleta de Dados e o Levantamento de Requisitos formam o alicerce de qualquer projeto. Um requisito é, essencialmente, uma condição ou capacidade que o sistema deve possuir. Eles são divididos entre:

- **Requisitos Funcionais:** que descrevem as ações e comportamentos do sistema (como "emitir um relatório de vendas")

Identificador Único

Tabela 01

ID	Nome
RF001	Consulta se aluno está presente
RF002	Inclui presença para aluno
RF003	Alterar presença em caso de falta
RF004	Consulta a turma que aluno está matriculado
RF005	Alterar para aluno desistente
RF006	Exclui aluno com falta consecutiva

Nome do RF (especificação do que o RF faz)

Sempre que for especificar os Requisitos Funcionais, procure colocar no nome da ação o que o sistema “FAZ”. Para a declaração do RF não existe um padrão, porém é adotada esta estrutura na maioria das empresas. Contudo, Pressman (1995), apud PUC 2016, destaca que existe um mito a respeito dos requisitos, afirmando que muitos clientes imaginam que os requisitos podem ser alterados com facilidade, por acharem que o desenvolvimento de software é flexível.

- **Requisitos Não Funcionais:** que tratam de aspectos de qualidade e restrições, como segurança, desempenho e facilidade de uso.

O processo de descobrir esses requisitos é chamado de elicitación, e exige que o profissional utilize diferentes estratégias para extrair informações que, muitas vezes, nem o próprio cliente sabe expressar com clareza.

Como destaca Sommerville (2003), requisitos bem definidos são a base para o sucesso de qualquer projeto de software, pois permitem alinhar expectativas, planejar entregas realistas e evitar surpresas durante o desenvolvimento. Compreender profundamente o conceito de requisito é, portanto, um passo fundamental para quem deseja atuar de forma eficiente e valorizada no mercado de desenvolvimento de sistemas.

Para realizar esse trabalho com precisão, o analista dispõe de uma caixa de ferramentas composta pelos seguintes métodos de levantamento de requisitos:

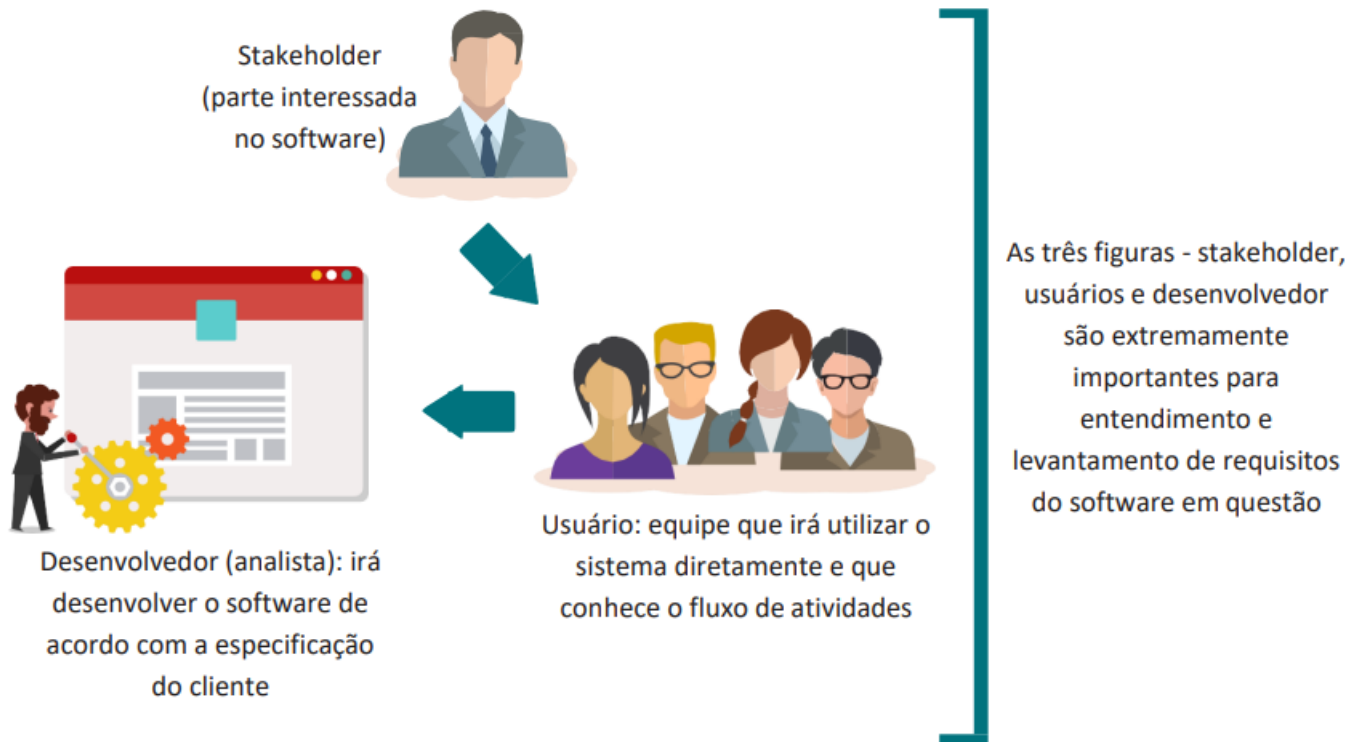
- **Entrevistas:** É a técnica mais direta, permitindo uma conversa aberta com os interessados (stakeholders). É ideal para entender fluxos complexos e captar nuances que documentos escritos não transmitem.

- **Questionários:** Úteis para coletar informações de um grande volume de pessoas simultaneamente. São eficazes para obter dados estatísticos e identificar padrões de comportamento em diferentes departamentos.
- **Observação de Campo:** Consiste em acompanhar o usuário em sua rotina real de trabalho. Muitas vezes, o usuário esquece de mencionar passos importantes por considerá-los óbvios, e a observação permite ao analista identificar essas lacunas.
- **Brainstorming e Reuniões de Grupo:** Promovem a colaboração entre diferentes áreas da empresa. O objetivo é gerar ideias e resolver conflitos de interesses logo no início do projeto.
- **Prototipagem:** Envolve a criação de esboços ou telas interativas simples. É uma forma visual de validar se o que o analista entendeu corresponde ao que o usuário espera, reduzindo drasticamente os erros de interpretação.



Imagem 3: Métodos de levantamento de requisitos.

A escolha do método adequado depende do tempo disponível, do perfil dos envolvidos e da complexidade do problema. O objetivo final dessa fase é garantir que o escopo do projeto esteja bem definido e que todos os envolvidos tenham a mesma visão sobre o que será entregue. Afinal, corrigir um erro de entendimento na fase de análise custa significativamente menos do que alterar um software que já está em fase de programação ou testes. Vamos imaginar o seguinte cenário: Há um ano, o Senhor Martins montou uma empresa de comércio eletrônico, porém o controle de seu estoque é realizado manualmente em planilhas de Excel e livro caixa. Com este processo, o Sr. Martins vem perdendo diariamente vendas importantes devido à falta de controle de seus produtos. Com base nesse cenário, vamos criar um grupo de desenvolvedores?



Depois de conhecer algumas técnicas para levantar informações, vamos entender as principais atividades da Engenharia de Requisitos, que ajudam a analisar, registrar e validar o que foi identificado. Conheça essas atividades a seguir:

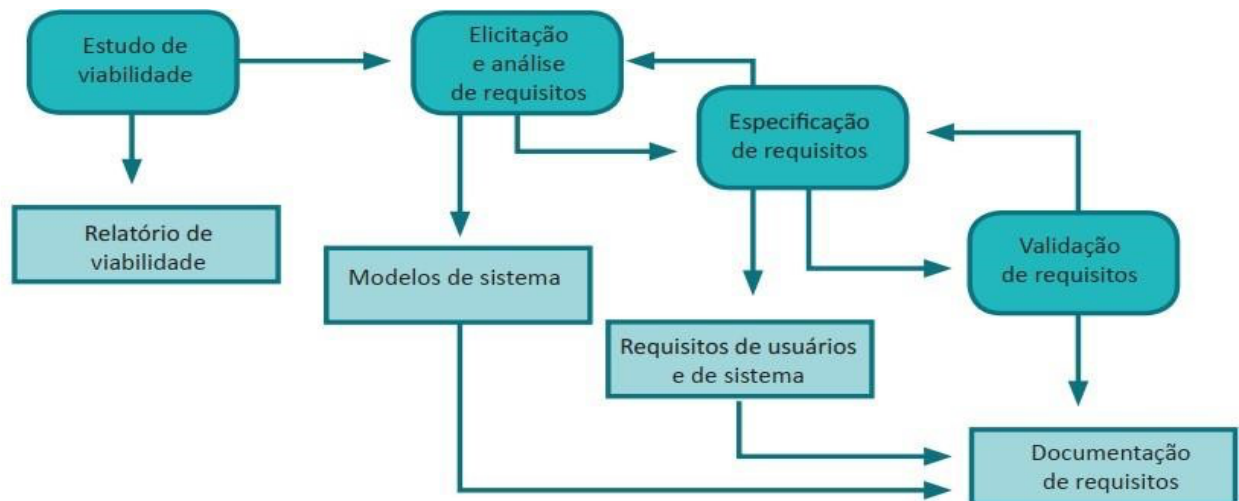
- **Estudo de viabilidade (Relatório de Viabilidade)** – verifica se as informações coletadas possibilitam implementar o desenvolvimento do software. É importante deixar claro para o cliente o que pode ou não ser feito no software para não criar uma expectativa falsa.

- **Elicitação e análise de requisitos (Modelo de sistema)** - neste momento, conseguimos identificar os detalhes para o desenvolvimento do software. A entrevista é uma das ferramentas que se pode utilizar para o cliente e/ou usuário relatar o fluxo de sua rotina de trabalho e as necessidades do processo como um todo.

- **Especificação de requisitos (Requisitos de usuário e de sistema)** – descrições necessárias para as funções essenciais no sistema.

- **Validação de requisitos (Documentação de requisitos)** – todos os processos de levantamento de requisitos devem ser documentados e qualquer mudança sugerida ou realizada no projeto também devem ser citadas, desta forma o cliente consegue validar todos os processos com clareza.

Os requisitos da engenharia de processo



Veja que as setas apontadas para cada processo indicam que a comunicação deve ser constante e que a qualquer momento pode-se realizar uma nova análise, sempre atento ao tempo de entrega do Software. Entretanto, antecedendo a etapa de projeto da arquitetura de software, há a necessidade de fazer o levantamento dos requisitos do sistema.

Erros comuns durante o levantamento de requisitos

No cotidiano de quem atua com análise de sistemas, é bastante comum se deparar com projetos que sofrem atrasos, retrabalho e até insatisfação do cliente — tudo por causa de falhas no levantamento de requisitos. Imagine uma equipe que, por pressa ou excesso de confiança, pula etapas importantes da conversa com o cliente e acaba assumindo que já entendeu tudo o que precisa ser feito. O resultado, muitas vezes, é um sistema que não resolve os problemas reais ou deixa de atender demandas cruciais.

Esses erros acontecem por diversos motivos: falta de escuta ativa, pouca participação dos usuários, comunicação falha entre áreas técnicas e de negócio, ou mesmo porque o analista não registra detalhadamente as informações obtidas. Além disso, é comum subestimar a importância dos requisitos não funcionais como desempenho, segurança e usabilidade, focando apenas nas funcionalidades do sistema.

Para evitar esses problemas, algumas estratégias são fundamentais: manter o cliente envolvido em todas as etapas do projeto, utilizar diferentes métodos de coleta (como entrevistas, observação e prototipagem), documentar cada requisito de forma clara e revisá-los com frequência junto a todos os envolvidos. Também é essencial validar os requisitos antes de iniciar o desenvolvimento, garantindo que todos tenham a mesma compreensão do que será entregue.

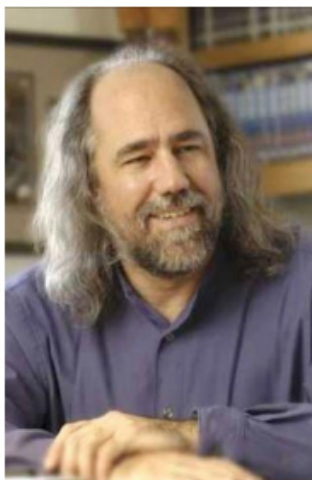
Como reforça Soares (2004), investir tempo na coleta e validação dos requisitos reduz significativamente falhas e retrabalho, tornando o processo de desenvolvimento mais eficiente e satisfatório para todos os envolvidos.

Modelagem Ágil e UML – Casos de Uso

Ao avançar no desenvolvimento de sistemas, você logo percebe que não basta apenas levantar e documentar requisitos: é preciso **organizar, visualizar e comunicar** essas informações de forma clara para toda a equipe e para o cliente. É aí que entram as metodologias ágeis e as ferramentas de modelagem, como **a UML, oferecendo caminhos para tornar o processo mais adaptável, transparente e eficiente.**

O levantamento e a análise de **requisitos ajudam a compreender o que o sistema precisa oferecer.** Em projetos que utilizam Scrum, essas necessidades **podem contribuir para a criação e o refinamento dos itens do Product Backlog,** que organiza o trabalho necessário para melhorar o produto. Para representar diferentes aspectos de um sistema, podemos utilizar diagramas da **UML (Unified Modeling Language, ou Linguagem Unificada de Modelagem).** O Diagrama de Casos de Uso apresenta funcionalidades e interações com atores externos, enquanto o Diagrama de Classes representa aspectos da estrutura do sistema, como classes, atributos e operações.

A UML surgiu na década de 1990 a partir do trabalho de Grady Booch, James Rumbaugh e Ivar Jacobson. Eles reuniram conceitos de métodos de modelagem orientada a objetos já existentes, como o método de Booch, a OMT (Object Modeling Technique), de Rumbaugh, e a OOSE (Object-Oriented Software Engineering), de Jacobson. Dessa colaboração resultou uma linguagem de modelagem utilizada para representar diferentes aspectos de sistemas de software.



Grady Booch



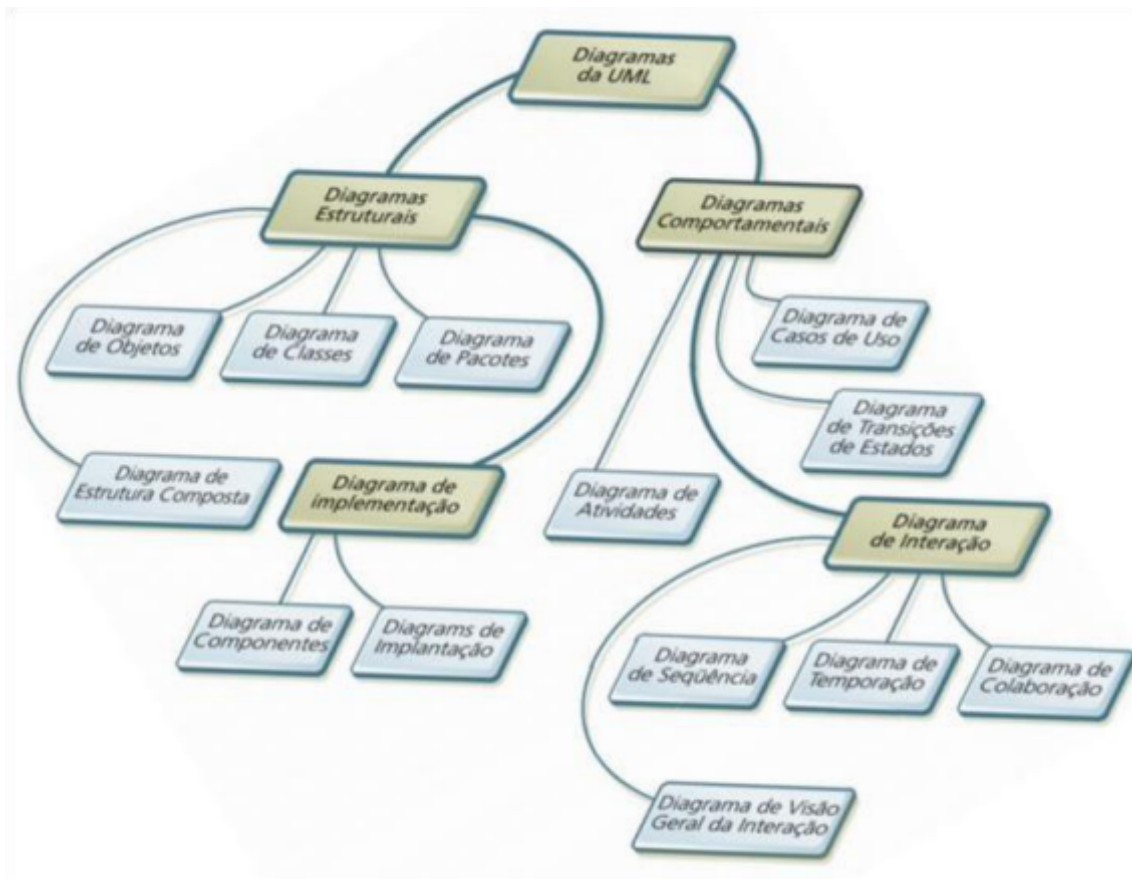
Ivar Jacobson



James Rumbaugh

Criadores da UML. Fonte imagens: <http://www.fabiobmed.com.br/booch-rumbaugh-jacobson-e-a-uml-unified-modeling-language/>

Por oferecer ferramentas que podem ser utilizadas em todas as fases do desenvolvimento de software, a UML acabou se tornando padrão de desenvolvimento de software orientado a objetos. Existem **13 diagramas na UML 2.0,** os quais são divididos em quatro grupos, de acordo com o tipo de análise que os modelos gerados por sua utilização possibilitam. São esses **os grupos: diagramas estruturais, diagramas comportamentais, diagramas de interação e diagramas de implementação.** Veja o gráfico a seguir:



Diagramas definidos pela UML 2.0

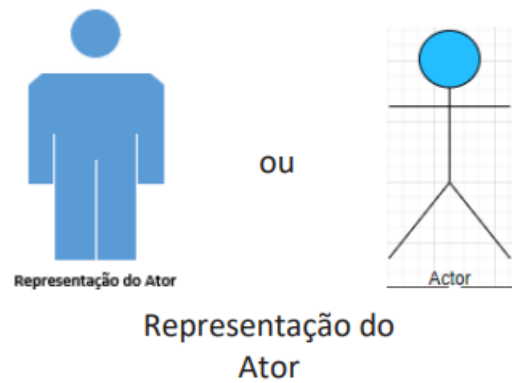
Assista ao vídeo “UML é importante para você?” do Canal DevMedia.

Para conhecer os diagramas da UML, acesse o artigo: Como eu uso UML para desenvolver sistemas? do Canal DevMedia.

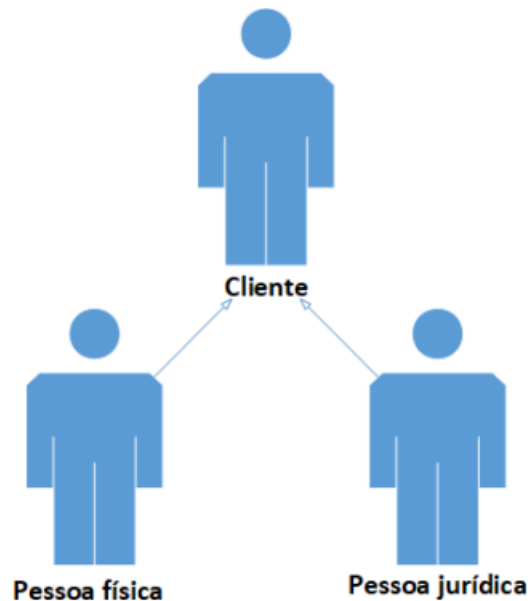
Disponível em <https://www.devmedia.com.br/modelagem-desistemas-atraves-de-uml-uma-visao-geral/27913>. Acessado em 15/03/2026.

Embora a UML possua diferentes tipos de diagramas, nesta agenda vamos estudar o Diagrama de Casos de Uso, que ajuda a representar as funcionalidades de um sistema e suas interações com atores externos. Principais componentes do Diagrama de Caso de Uso são: o cenário, os atores, os casos de uso e os relacionamentos.

Um Ator em um Caso de Uso representa uma entidade externa ao sistema que interage de alguma forma com um Caso de Uso. Normalmente, um ator é uma pessoa, um dispositivo de hardware ou um outro sistema. No Diagrama de Caso de Uso, o ator é representado como na imagem:



Podemos também utilizar o conceito de **Generalização de Atores**:



Vamos criar um exemplo de um cenário para entender melhor a notação de um Diagrama de Caso de Uso:

“O Cliente chega ao caixa eletrônico de um banco, efetua um saque em sua conta corrente, emite o extrato da conta e efetua um pagamento de um boleto. O sistema bancário controla as operações de saque, de emissão de extrato e do pagamento do boleto. O administrador do sistema alimenta o caixa com as cédulas monetárias (dinheiro).

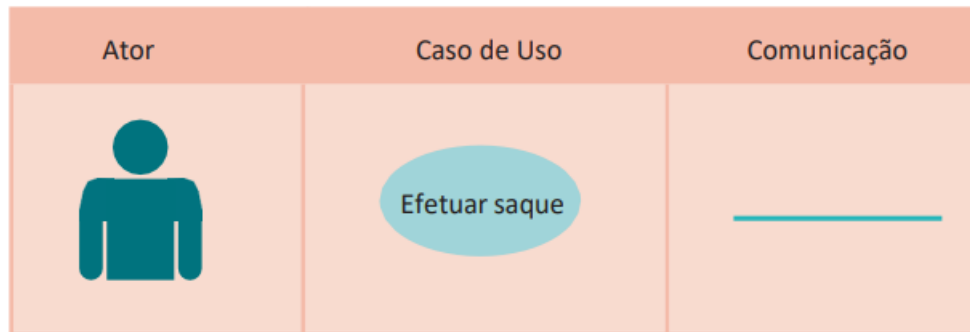
Com esse cenário simples podemos começar a criar nosso diagrama. Inicialmente vamos definir nossos atores:

- Cliente
- Sistema Bancário
- Funcionário do Banco

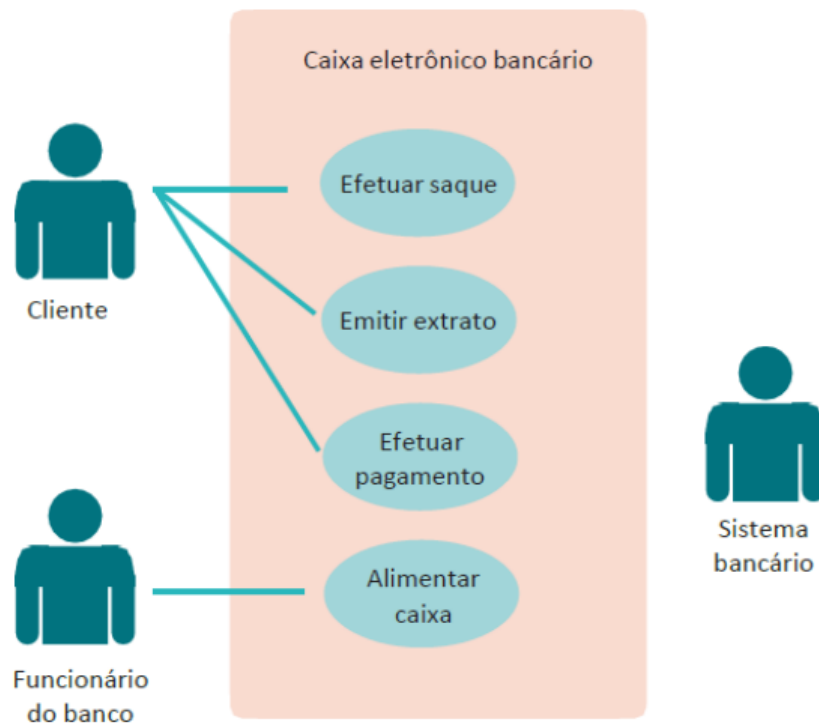
Analisando a descrição do cenário, podemos identificar as ações dos atores:

- Cliente: Efetua saque; Emite extrato; Efetua Pagamento.
- Sistema Bancário: Controla operação de efetuar saque; Controla operação de emitir extrato; Controla operação de efetuar pagamento.
- Funcionário do Banco: Alimenta o caixa.

Veja as representações de cada elemento no quadro “Elementos do Diagrama de Caso de Uso”:



Agora vamos construir o diagrama de caso de uso:

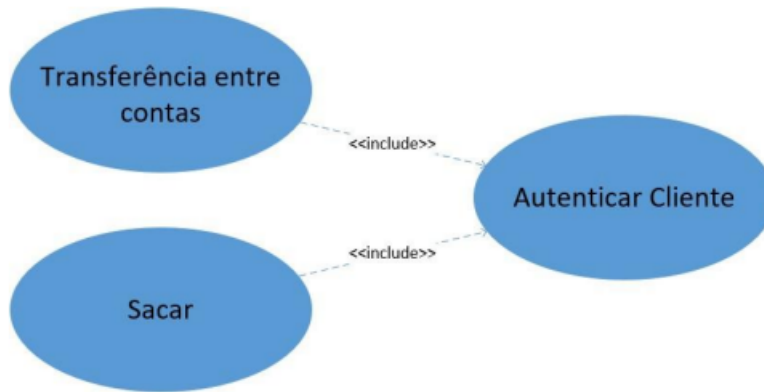


Como se pode observar, o diagrama de caso de uso é composto por desenhos simples que descrevem as funcionalidades do sistema e as interações dos atores com elas.

Existem ainda os conceitos de <> **Include** e <> **Extend** que podem melhorar ainda mais as especificações das ações no diagrama.

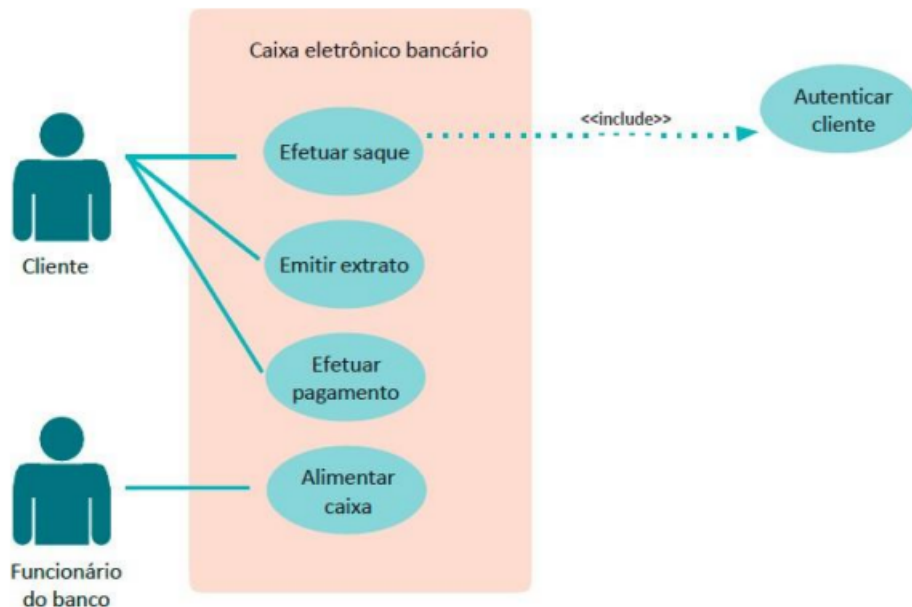
<> Include

Uma relação de **Inclusão** de um **Caso de Uso A** com um **Caso de Uso B** indica que uma instância do **Caso de Uso A** deve incluir o comportamento especificado para o **Caso de Uso B**.



<> Extends

Uma relação de Extensão de um **Caso de Uso A** com um **Caso de Uso B** indica que uma instância do **Caso de Uso B** pode incluir - sujeito ao atendimento de certas condições - o comportamento especificado para o **Caso de Uso A**. Com isso, podemos aprimorar o diagrama inserindo um novo caso de uso "Autenticar Cliente", que será utilizado no caso de uso "Efetuar saque". Por questão de segurança, o sistema precisa confirmar a autenticidade do cliente.



O diagrama de Caso de Uso é muito importante para **auxiliar na definição dos Requisitos**, na Comunicação com os Clientes por não exigir conhecimentos técnicos e para **Delimitar o Contexto do Sistema**.



VOCÊ NO COMANDO

Vamos criar um diagrama de Caso de Uso para um Sistema de Biblioteca Escolar com o seguinte contexto:

A biblioteca de uma escola possui muitos usuários, dentre eles: professores e alunos que realizam empréstimos e devoluções de exemplares com o atendente, funcionário da biblioteca. Se o usuário não estiver cadastrado, ele deve informar seus dados pessoais como: nome completo, telefone e endereço. No caso do professor, deve ser informada também a sua formação e no caso do aluno, deve ser informado o nome do curso no qual está matriculado.

- No empréstimo e/ou devolução os usuários podem realizar a retirada ou devolução dos livros desejados, e o atendente registra a operação com os seguintes dados: data do empréstimo, data da devolução e os exemplares desejados.
- A biblioteca possui muitos exemplares de um livro, cujos atributos são: código do livro, editora e título.
- É responsabilidade do atendente manter os cadastros dos professores e alunos atualizados. Se você fez o rascunho do seu Diagrama de Caso de Uso, é hora de conferir se você acertou.

Para iniciar o levantamento das informações devemos começar identificando os atores do caso de uso, neste contexto temos:



Neste diagrama, o atendente, o professor e o aluno são identificados como atores porque são externos à situação do cenário e não representam nenhuma funcionalidade e também não se comportam como um requisito do sistema.

Aplicando o conceito de herança, podemos simplificar os atores Aluno e Professor:



As ações realizadas pelos atores são:

- alunos, professores e atendente: realizar devolução e realizar empréstimo.
- atendente: manter professores e alunos.
- atendente: manter exemplares.

De acordo com os conceitos vistos podemos aplicar a generalização de atores entre alunos e professores e criar um ator usuário pois, ambos têm atributos em comum como: nome completo, telefone e endereço.

Para representar o caso de uso, lembre-se de que os bonecos são os atores, as elipses são os casos de uso e as linhas representam a comunicação entre ator e caso de uso.

A imagem apresenta o Diagrama de Caso de Uso, de acordo com análise do contexto da Biblioteca Escolar.

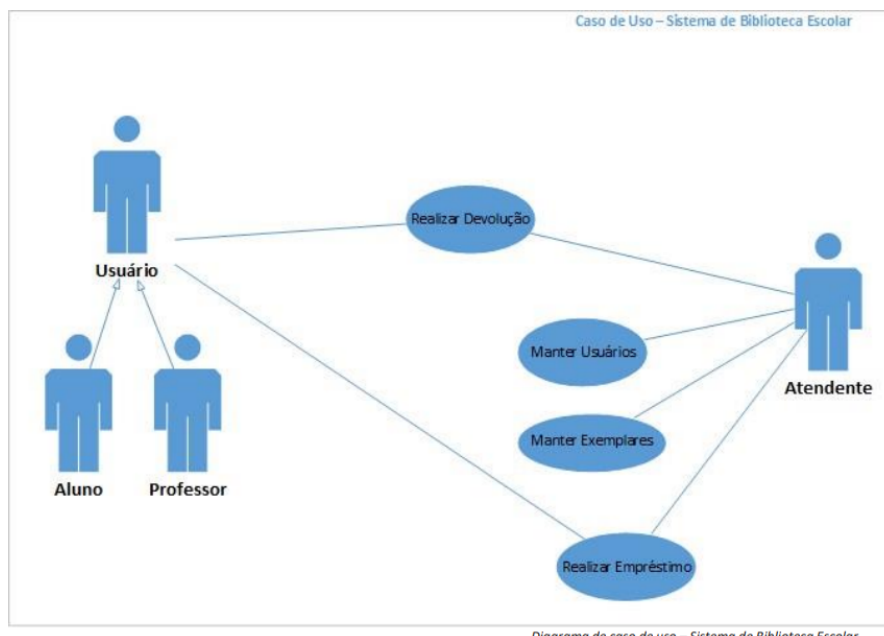


Diagrama de caso de uso – Sistema de Biblioteca Escolar

Referências Bibliográficas

SOMMERVILLE, Ian. Software Engineering. 6. ed. Harlow: Addison-Wesley, 2000.

ROYCE, Winston W. Managing the development of large software systems: concepts and techniques. In: PROCEEDINGS OF IEEE WESCON. Los Angeles: IEEE, 1970.

PUC. Métodos de desenvolvimento de produtos de interfaces gráficas digitais: da abordagem em cascata à ágil. Disponível em: https://www.maxwell.vrac.puc-rio.br/21839/21839_3.PDF. Acesso em: 14 set. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 8. ed. São Paulo: Addison-Wesley, 2007.

MANIFESTO ÁGIL. Disponível em: <https://agilemanifesto.org/iso/ptbr/manifesto.html>. Acesso em: 14 set. 2026.